

# Lista de Exercícios: Lógica de Programação em C

---

**Instruções:** Para todos os exercícios abaixo, declare as variáveis e atribua valores decorrentes de entradas do usuário, usando `scanf()`. O objetivo é praticar a estrutura da linguagem e a lógica dos comandos.

**Envio:** O envio dessa lista deve ser um único arquivo compactado **.zip** ou **.rar** contendo todos os arquivos **.c** escritos. O nome do arquivo compactado deve ser seu nome e sobre, seguido de sua matrícula. Ex.: **bruno\_silva\_1352853.zip**.

**Ferramentas:** Para testar os códigos em um ambiente "seguro", recomendo a ferramenta [onlinegdb](#) ou [compilador online da programiz](#), ferramenta online uteis para estudo de C.

**IA:** O uso de IA é interessante, desde que ajude no processo do aprendizado, tendo em mente que a avaliação é manuscrita, então é importante entender/aprender os conceitos e estrutura dos códigos desenvolvidos.

## Recapitulação: Funções em C

Em programação, uma **função** é um bloco de código independente projetado para realizar uma tarefa específica. Pense nela como uma "máquina" ou um operário especializado: você fornece os ingredientes (parâmetros), ela executa a lógica (variáveis, condicionais, laços) e devolve o produto final (retorno).

O uso de funções ajuda a organizar o código, evitar repetições desnecessárias e facilitar a manutenção.

### A Estrutura de uma Função

Toda função em C possui uma assinatura e um corpo, seguindo a sintaxe abaixo:

```
tipo_de_retorno nome_da_funcao(tipo_parametro1 nome_parametro1, tipo_parametro2
nome_parametro2) {
    // Escopo local: as variáveis criadas aqui só existem dentro da função

    // Lógica: aqui entram as condicionais (if/switch) e laços (for/while)

    return valor_de_retorno; // Opcional se o retorno for 'void'
}
```

### Componentes Principais

- **Tipo de Retorno:** É o tipo de dado que a função vai devolver para quem a chamou (ex: `int`, `float`, `double`). Se a função não precisar devolver nada, usamos o tipo `void`.
- **Nome da Função:** O identificador usado para chamá-la no código (ex: `calcular_soma`).
- **Parâmetros (ou Argumentos):** As variáveis de entrada que a função precisa para trabalhar. Eles ficam entre parênteses. Se não houver entradas, deixamos os parênteses vazios ou com `void`.
- **Corpo da Função:** Todo o código que fica entre as chaves `{ }`.
- **Comando `return`:** Finaliza a execução da função e envia o resultado de volta.

---

### Exemplo Prático

Veja como uma função processa variáveis e condicionais internamente e devolve um resultado:

```
// Esta função recebe um número inteiro e verifica se ele é par
int eh_par(int numero) {
    // Uso de condicional e operadores matemáticos
    if (numero % 2 == 0) {
        return 1; // Retorna 1 para "Verdadeiro" (é par)
    } else {
        return 0; // Retorna 0 para "Falso" (é ímpar)
    }
}
```

**Nota de Escopo:** Lembre-se de que as variáveis criadas dentro de uma função (incluindo seus parâmetros) são **locais**. Elas nascem quando a função é chamada e deixam de existir assim que ela encontra o comando **return** ou chega ao fim de suas chaves.

## Questões Progressivas (1 a 10)

*As questões abaixo aumentam progressivamente o nível de complexidade lógica e a integração dos conceitos prévios.*

**Questão 1 (Fácil - Laço **for** e Variáveis):** Crie um algoritmo que receba um número inteiro positivo N como parâmetro. Utilizando um laço **for**, a função deve calcular e retornar a soma de todos os números inteiros de 1 até N.

**Questão 2 (Fácil - Laço **while** e Condicionais):** Crie um algoritmo que receba um número inteiro positivo limite como parâmetro. Utilizando um laço **while**, a função deve percorrer todos os números de 1 até esse limite e retornar a quantidade de números que são estritamente pares.

**Questão 3 (Fácil/Médio - Laço **do-while** e Condicionais):** Crie um algoritmo que receba um número inteiro positivo N. Utilizando um laço **do-while**, a função deve realizar uma contagem regressiva a partir de N até 0. No entanto, utilizando uma estrutura condicional, a função deve ignorar os números múltiplos de 5. Retorne o total de números efetivamente processados.

**Questão 4 (Médio - Laço **for** e Estrutura **switch**):** Crie um algoritmo que receba um número inteiro N representando uma quantidade de dias. Utilizando um laço **for** que itere de 1 até N, utilize uma estrutura **switch** para determinar o dia da semana atual da iteração (assumindo que 1 é Segunda-feira, 2 é Terça-feira, até 7 que é Domingo, e depois repetindo o ciclo). A função deve contar e retornar apenas quantos dias caíram em um "Fim de Semana" (Sábado ou Domingo).

**Questão 5 (Médio - Laço **while** e Acumuladores):** Crie um algoritmo que receba um número inteiro positivo N e calcule o seu fatorial utilizando um laço **while**. Lembre-se de tratar adequadamente o caso base onde N = 0 utilizando uma estrutura condicional antes ou durante o laço.

**Questão 6 (Médio - Laço **for** com Múltiplos Parâmetros):** Crie um algoritmo que receba três números inteiros: **inicio**, **fim** e **incremento**. Utilizando um laço **for**, a função deve percorrer o intervalo que vai de **inicio** até **fim**, saltando de acordo com o valor de **incremento**. A função deve retornar a soma de todos os números visitados nesse intervalo.

**Questão 7 (Médio/Difícil - Laço **do-while** e Operadores Matemáticos):** Crie um algoritmo que receba um número inteiro positivo N. Utilizando um laço **do-while**, a função deve encontrar e somar todos os divisores estritos desse número (ou seja, todos os números menores que N pelos quais N é divisível com resto zero). Retorne a soma final.

**Questão 8 (Médio/Difícil - Laço `for` e Interrupção por Limite):** Crie um algoritmo que receba dois números inteiros positivos: um número limite  $N$  e um valor máximo  $M$ . Utilizando um laço `for` de 1 até  $N$ , a função deve somar apenas os números ímpares. Caso a soma acumulada ultrapasse o valor máximo  $M$ , o laço deve ser interrompido imediatamente (usando `break`) e a função deve retornar o número atual da iteração onde o limite foi estourado. Caso termine o laço sem estourar, retorne -1.

**Questão 9 (Difícil - Laço `for` com Lógica Dinâmica via `switch`):** Crie um algoritmo que receba um número inteiro positivo  $N$ . Um laço `for` deve iterar de 1 até  $N$ . Dentro do laço, uma estrutura `switch` deve avaliar o resto da divisão do contador atual por 3 (`contador % 3`). Se o resto for 0, adicione o valor do contador a uma variável acumuladora; se o resto for 1, subtraia o valor do contador desta variável; se o resto for 2, multiplique a variável acumuladora por 2. A variável acumuladora deve iniciar em 10. Retorne o valor final acumulado após o término do laço.

**Questão 10 (Identificação de Primalidade com Laço Único `for`):** Crie um algoritmo que receba um número inteiro positivo  $N$  e determine se ele é um número primo. Utilizando um único laço `for` e estruturas condicionais, verifique se o número possui outros divisores além de 1 e dele mesmo. Exiba se o número é primo ou não.